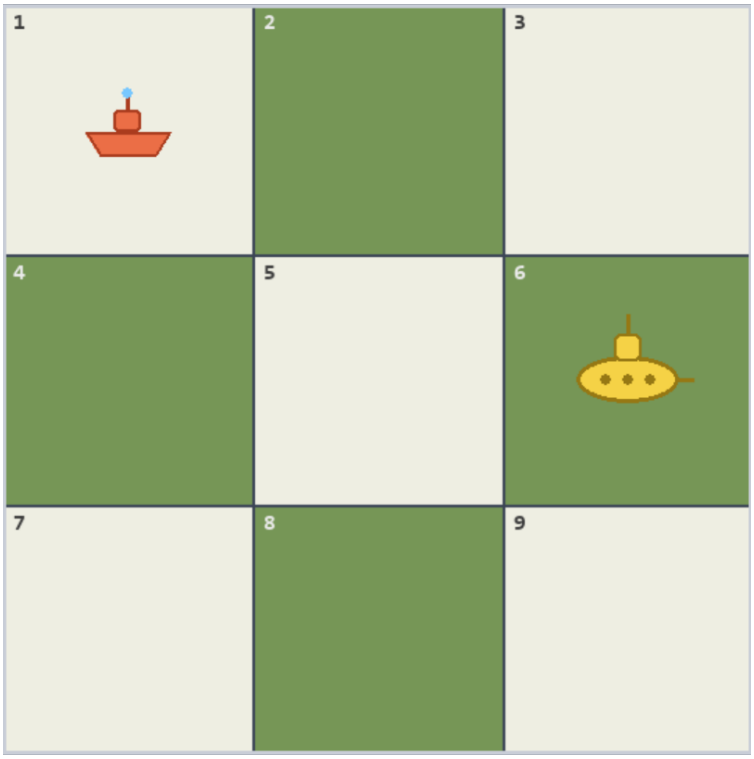
Constrained Bayesian Search for a Moving Target

Miquel Àngel Llauger Suau Lluc Segura Lladó

Final Project: Bayesian Statistics & Probabilistic Programming



Contents

1 Motivation and connection with the course

A central idea in Bayesian statistics is that uncertainty is represented by a probability distribution and updated when new evidence is observed. In the spatial search exercise we studied as a second deliverable, the Palomares-inspired bomb recovery problem, the search region is divided into cells and the unknown target location is described by a prior distribution $\pi = (\pi_1, \ldots, \pi_N)$. If a search of cell j fails, Bayes' rule reduces the posterior mass in that cell according to the probability of detection and redistributes the remaining probability over the other cells.

That model is useful when the target is stationary. However, many practical search problems are dynamic: a ship can drift, a lost hiker can move, and a submarine may deliberately change position. Moreover, the searcher is usually constrained by physics. A rescue boat or aircraft cannot search any cell at the next time step; it can only move to nearby cells. These two features make the problem more complex than the static case, because the best current search may not be the cell with the largest posterior probability. It may be better to move to a cell that gives access to a more promising region later.

Our project is based on the constrained moving-target search formulation of Eagle [1]. The course connection is the Bayesian update of beliefs after observations. The new element is that these beliefs become the state variable of a sequential decision problem. In modern terminology, this is a POMDP (Partially Observable Markov Decision Process): the real target location is hidden, but the posterior distribution over possible locations is observed and updated recursively.

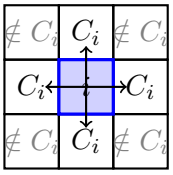
2 Model formulation

Let the search area be divided into N cells $C = \{1, \ldots, N\}$ and let time be discrete, $t = 1, \ldots, T$. The target location at time t is a random variable $Z_t \in C$. Its movement is modeled as a Markov chain with transition matrix $P = [p_{ij}]$, where

$$p_{ij} = \Pr(Z_{t+1} = j \mid Z_t = i), \qquad \sum_{j=1}^{N} p_{ij} = 1. \tag{1}$$

Thus, the target's next position depends on its current cell but not on earlier history.

The searcher is also located in one cell per period. If the previous searched cell was i, the next searched cell must belong to a feasible set $C_i \subseteq C$. For example, on a grid C_i may contain the current cell and its horizontal and vertical neighbours. This constraint is what transforms the problem from a simple ranking of cells into a path-planning problem.



Example with only horizontal/vertical moves:
the next search must remain inside C_i.

If the searcher searches cell j and the target is actually in j, the detection probability is $q_j \in [0, 1]$. If the target is in any other cell, detection probability is zero. Let

$$r(t) = \big(r_1(t), \ldots, r_N(t)\big) \tag{2}$$

be the belief distribution at the beginning of period t, conditional on all previous searches having failed. Here $r_k(t) = \Pr(Z_t = k \mid \text{previous failures})$.

Bayesian posterior update

Suppose that at time t we search cell j and do not detect the target. The probability of this failed observation is $1 - q_j r_j(t)$. Before the target moves, Bayes' rule gives

$$r'_k(t) = \begin{cases} \dfrac{r_k(t)}{1 - q_j r_j(t)}, & k \neq j, \\ \dfrac{r_j(t)(1 - q_j)}{1 - q_j r_j(t)}, & k = j. \end{cases} \tag{3}$$

The searched cell loses probability mass because a failed search is evidence against the target being there. After this update, the target moves according to P, so

$$r_k(t+1) = \sum_{m=1}^{N} r'_m(t) p_{mk}. \tag{4}$$

In compact form, define D_j as the diagonal matrix whose jth diagonal element is $1 - q_j$ and whose other diagonal elements are 1. Then

$$T_j(r) = \frac{r D_j P}{1 - q_j r_j} = \frac{r P_j}{1 - q_j r_j}, \tag{5}$$

where $P_j = D_j P$ is the transition matrix with row j multiplied by $1 - q_j$. The map T_j is the Bayesian filter: it transforms the current belief into the next belief after an unsuccessful search of cell j.

Let $V_n(r, i)$ be the maximum probability of detecting the target during the next n periods, given current belief r and previous searcher position i. If we choose a feasible next cell $j \in C_i$, the immediate probability of detection is $q_j r_j$. If detection fails, which has probability $1 - q_j r_j$, the belief becomes $T_j(r)$ and the searcher is now at j. Thus, we define

$$V_n(r, i) = \max_{j \in C_i} \left\{ q_j r_j + (1 - q_j r_j) V_{n-1}(T_j(r), j) \right\}, \tag{6}$$

with boundary condition $V_0(r, i) = 0$.

Intuitively, this series of numbers $V_n(r, i)$ tell us which are the most likely paths of length n that guide us to the submarine, for a current belief and position. Here, we sum the probability of finding the submarine at cell C_j, with the probability of finding it in some path that goes trhough cell C_j conditional to it not being there. That is exactly what solves the problem, we fix a horizon, in this case $n = 10$, and search the space of paths.

3 Eagle's solution

The computational difficulty is that the number of possible paths grows exponentially. There are at most 4 possible moves at each time step, so the number of possible paths grows as $4^n \approx 10^6$ in our case. The keypoint of Eagle's paper [1] is adressing this problem with Dinamic Programming and a pruning strategy that makes it feasible. We will just draw the outline of the strategy, rather than dive in its details.

Sketch of the strategy

A key theoretical result is that V_n is piecewise linear and convex in the belief r. More precisely, for every horizon n and previous cell i, there exists a finite set of vectors $A(n,i)$ such that

$$V_n(r,i) = \max_{a \in A(n,i)} ra. \tag{7}$$

For one remaining period, $A(1,i) = \{q_j e_j : j \in C_i\}$, where e_j is the jth standard basis vector. By induction, the vector update rule is

$$A(n,i) = \bigcup_{j \in C_i} \{q_j e_j + P_j a_j \mid a_j \in A(n-1,j)\}. \tag{8}$$

Each vector represents the detection probability of a particular future search path conditional on each possible starting target cell. The optimal path for a given prior is found by selecting the vector with the largest dot product ra.

The computational difficulty is that the number of vectors can grow exponentially. If each cell has about M feasible moves, then $|A(n,i)|$ is roughly M^n. Many of these vectors are dominated. A vector a can be removed if another vector b satisfies $b_k \geq a_k$ for all k and is strictly better for at least one k.

4 Reproduction of the paper's experiment

We first reproduced the numerical example from Eagle [1]. The original problem uses a 3×3 grid with cells numbered as

1	2	3
4	5	6
7	8	9

The searcher may stay in the previously searched cell or move to a cell that shares a side with it. For example, from the central cell we have $C_5 = \{2,4,5,6,8\}$, while from the corner cell 1 we have $C_1 = \{1,2,4\}$. In this example detection is perfect: if the target is in the searched cell, it is detected with probability one. Therefore, $q_j = 1$ for all cells j.

The target movement in Eagle's example is also defined on this grid. At each transition, the target remains in its current cell with probability 0.4 and moves to one of its side-adjacent cells with probability $0.6/m_j$, where m_j is the number of adjacent cells of cell j. The initial distribution is concentrated in cell 9:

$$r = (0,0,0,0,0,0,0,0,1), \tag{9}$$

and the search horizon is $T = 10$ periods.

We implemented the dynamic programming algorithm with the pruning/dominance method in the notebook and reproduced this example. For this specific problem, we obtained the same detection probabilities and the same type of optimal paths reported in Eagle's paper. Since some starting cells have several optimal paths, the table includes only two examples when needed:

Starting cell	Max P_d	Num. optimal paths	Example optimal paths
1	0.7786	6	2369856585; 2569856585
2	0.8631	3	3698856585; 5698856585
3	0.8631	2	3698856585; 6698856585
4	0.8631	3	5698856585; 5896658565
5	0.8631	4	5698856585; 5896658565
6	1.0000	128715	any path beginning with 9
7	0.8631	2	7896658565; 8896658565
8	1.0000	128715	any path beginning with 9
9	1.0000	128715	any path beginning with 9

In the original paper, Eagle reports that the calculation took 19.3 CPU minutes on an IBM 3033. In our case, the reproduction took around five minutes in the notebook. This is reasonable, since the mathematical problem is the same but the available computing power has changed enormously over the last 50 years. For us, the main conclusion of this first reproduction was that our implementation was able to solve the problem correctly using the pruning/dominance method.

5 Further experiments

Once we had reproduced the published result, we used the same algorithm to carry out some further experiments. First, we changed the description of the target movement to simulate a faster target. In this new model, the target stays in the same cell with probability 0.2 and moves to one of its adjacent cells with probability $0.8/m_j$. The corresponding transition matrix is then:

$$P_{\text{fast}} = \begin{pmatrix} 0.2 & 0.4 & 0 & 0.4 & 0 & 0 & 0 & 0 & 0 \\ 0.2667 & 0.2 & 0.2667 & 0 & 0.2667 & 0 & 0 & 0 & 0 \\ 0 & 0.4 & 0.2 & 0 & 0 & 0.4 & 0 & 0 & 0 \\ 0.2667 & 0 & 0 & 0.2 & 0.2667 & 0 & 0.2667 & 0 & 0 \\ 0 & 0.2 & 0 & 0.2 & 0.2 & 0.2 & 0 & 0.2 & 0 \\ 0 & 0 & 0.2667 & 0 & 0.2667 & 0.2 & 0 & 0 & 0.2667 \\ 0 & 0 & 0 & 0.4 & 0 & 0 & 0.2 & 0.4 & 0 \\ 0 & 0 & 0 & 0 & 0.2667 & 0 & 0.2667 & 0.2 & 0.2667 \\ 0 & 0 & 0 & 0 & 0 & 0.4 & 0 & 0.4 & 0.2 \end{pmatrix}. \tag{10}$$

We first applied the algorithm to this faster-target case using the same prior as before,

$$r = (0, 0, 0, 0, 0, 0, 0, 0, 1). \tag{11}$$

This means that, at the initial time, we still know that the target starts exactly in cell 9, but after that it spreads through the grid more quickly. We recomputed the algorithm with this new transition matrix, which describes the new target movement model. The results obtained for this newly defined problem are:

Starting cell	Max P_d	Num. optimal paths	Example optimal paths
1	0.7831	12	1256655855; 1258855055
2	0.8779	3	3658855655; 5658855655
3	0.8779	2	3658855655; 6658855655
4	0.8779	3	5658855655; 5856655855
5	0.8779	4	5658855655; 5856655855
6	1.0000	624	9652254455; 9652254558
7	0.8779	2	7856655855; 8856655855
8	1.0000	624	9652254455; 9652254558
9	1.0000	624	9652254455; 9652254558

Once obtained this table of results, we can immediately know which paths are the best to take depending on the cell where the search starts. The table also gives the maximum probability of finding the faster moving target during the corresponding path. An interesting result of this case is that, depending on the starting cell, there can be either hundreds or just a couple of different paths with the same optimal probability

After obtaining these results, we changed the prior distribution. This can be interpreted as a new sensor reading or an error in the first sensor signal: instead of knowing the exact initial position of the target, the sensor gives uncertain information over all cells. We used

$$r = (0.03, 0.04, 0.05, 0.08, 0.10, 0.14, 0.16, 0.18, 0.22). \tag{12}$$

Here, cell 9 is still the most likely cell, but the target is not assumed to be there with certainty.

One important advantage of this method is that the computed vector sets work for any prior distribution. Therefore, once the dynamic programming and pruning steps have been completed for a given movement model, changing the prior does not require solving the full problem again. We only need to evaluate the already computed vectors with the new prior, so the new result is obtained almost instantly. The obtained results are the following:

Starting cell	Max P_d	Num. optimal paths	Optimal search path
1	0.7983	1	4558855255
2	0.8105	1	5885565555
3	0.8145	1	6658855255
4	0.8166	1	7855655455
5	0.8270	1	8855655455
6	0.8240	1	9885525555
7	0.8270	1	8855654455
8	0.8270	1	8855655455
9	0.8270	1	8855655455

In this new scenario, where the prior is less strict, we can verify that the optimal probabilities go down in almost all cases, as expected, because we no longer know the initial target location exactly. The only exception is the case where the search starts in cell 1, where the probability is slightly higher than in the previous prior.

From the point of view of the method studied, the interesting result is that changing the prior we obtained the updated optimal paths and probabilities in less than a second.

The deliverable contains one notebook with the paper's reproduction and the further experiments we carried on, a *demo* folder and this report. The demo will be shown during the presentation to better visualize some scenarios of the studied problem.

References

[1] J. N. Eagle. "The Optimal Search for a Moving Target When the Search Path is Constrained." *Operations Research*, 32(5):1107–1115, 1984.